Architecture Defense

Soundcheck — AI Event-File Ingestion Pipeline

Steven Tran · Gauntlet AI Architecture Review · June 2026 · *diagram attached*

> **Thesis.** Event-production data arrives as an unstructured mess — riders, advance sheets, contracts, promoter emails, Slack threads, spreadsheets, calendar invites. This pipeline turns "*drop your files*" into **structured, validated, human-approved Event records**, capturing every run as training data. The design I'm defending is the *shape*: **two tools separated by two guardrails, over an observability loop.**

```
Event Files → Check AI (extract) → UEF (schema guardrail) → MCP (capability) → Validation (human-approval guardrail) → Platform → Event
```

Core problems

1. **Format chaos** — every venue and promoter sends data differently; hand-writing a parser per source doesn't scale.
2. **Garbage-in risk** — an LLM reading a contract can hallucinate a fee or headcount; that can't silently become a payment or a roster.
3. **Cost at scale** — ingestion is a core platform function, so per-file LLM cost is an architectural constraint, not an afterthought.
4. **Wasted signal** — every "AI proposed X, human corrected to Y" is a free gold label worth capturing.

Pipeline walkthrough

Event Files (input) — any source: PDFs, images, spreadsheets, `.ics`, `.eml`, Slack attachments. Bytes are staged *by reference* and never enter the model context.

Check AI (tool) — extracts structure per file. Hybrid-routed by type; `.ics` / `.eml` parsed in Go first. One fused classify+extract call per file — document tokens are never paid twice.

Universal Event Object — UEF (guardrail) — the single normalized schema every extraction must conform to. **The seam that decouples extraction from persistence:** the model emits UEF, the platform consumes UEF, and a new file format never touches the commit path.

Soundcheck MCP (tool) — 73 tools over the REST API. The agent can't touch the DB or raw API; every action is org/caller-scoped with destructive-action annotations. **Capability, not authority.**

Validation (guardrail) — (1) preview-before-commit is a *server* invariant (rejected unless `PREVIEWED`); (2) the confirm token binds a hash of the exact diff, so any re-merge auto-invalidates it and forces re-approval.

Platform → Event → fan-out — approved UEF becomes one Event spanning Timeline, People, Payments, Assets, Venue. Multi-file batches merge first with provenance, dedup, and conflict warnings.

Loop / Observability — every run captured as a gold pair (`extractedUef` → `committedUef`) plus per-call telemetry, prompt-versioned. The review→commit flow *is* the labeling loop.

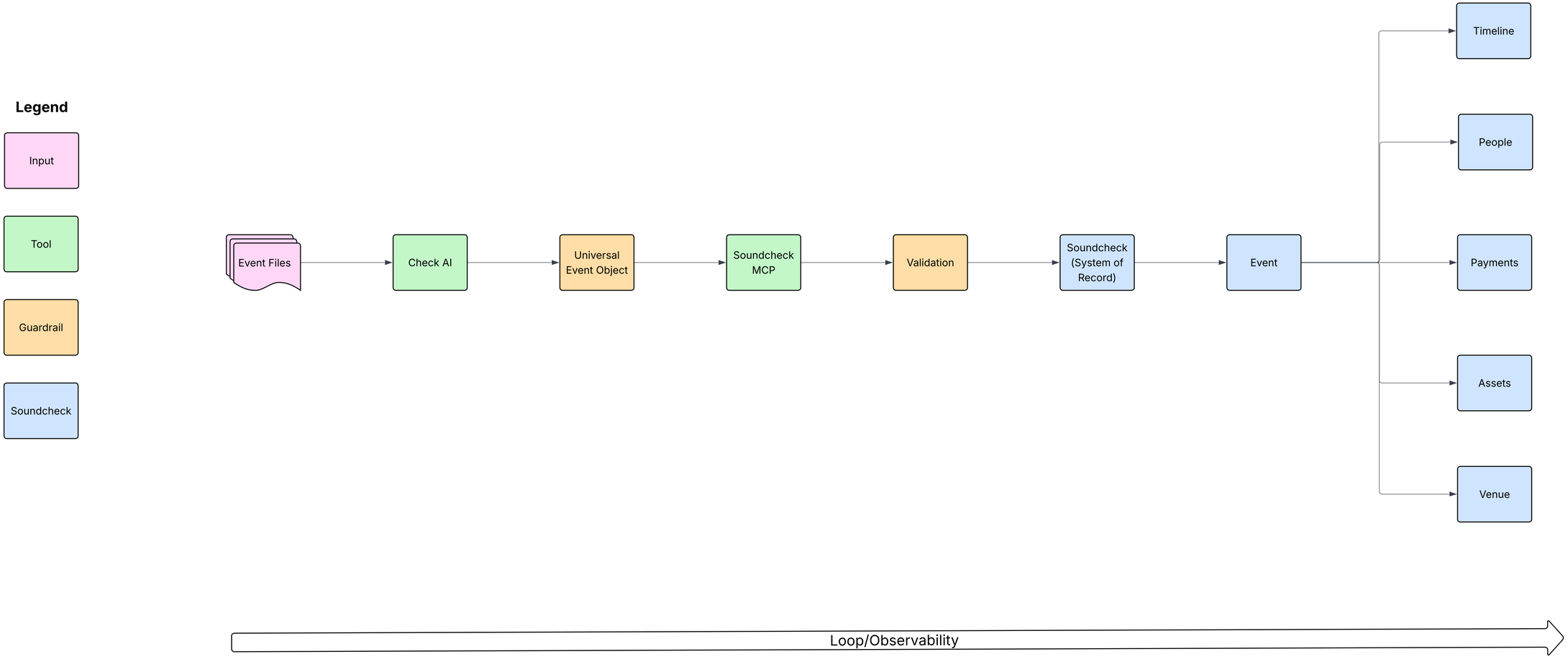
Key decisions & why

Decision	Why this over the alternative
UEF intermediate schema	vs. file→DB directly: decouples N formats from 1 commit path; constrains model output to a contract; new formats never touch persistence.
Everything through the MCP	vs. agent hits API/DB directly: one authz + confirmation chokepoint; file bytes never reach the model; new tools need zero agent wiring.
Two guardrails, not one	UEF = "is it well-formed?"; Validation = "did you mean to write this?" They catch different failures.
Preview-before-commit as a server invariant	vs. client-side check: a buggy or compromised client still can't commit unreviewed data — the guarantee lives in the API.
Token efficiency as architecture	Fused call/file, deterministic pre-parsing, tiered models, content caps. Cost-per-file is a tracked KPI.
Capture every run as training data	The human correction is a free, high-signal gold label; prompt-versioned so pairs stay comparable.

Known tradeoffs

- **Unsigned coordination markers** — the agent's "upload these files" signal is unsigned JSON, guarded only by a tool-name gate + batch re-validation with the user's bearer. The part I trust least.
- **Thin content-injection defense** — document bodies flow into prompts; body-level defense is regex probes. The real backstop is the human confirm gate, not model hardening.
- **Models fail ugly** — observed a light model hit a degenerate repetition loop on one doc. It failed closed (retryable, nothing committed), but extraction reliability isn't solved.
- **Rate-limiting is app-level per-IP** — no edge WAF on the MCP path; accurate only at a single instance.
- **Labeling loop isn't automated** — gold pairs are captured, but there's no annotation queue or eval-replay harness yet.

> **One-sentence defense.** Capability is separated from authority by an MCP chokepoint, structural validity from semantic approval by two guardrails, and nothing reaches the platform without a human approving the exact diff — so the worst an LLM failure can do is waste a retry, never write a bad payment.


Legend
Input
Tool
Guardrail
Soundcheck
Event Files
Check AI
Universal Event Object
Soundcheck MCP
Validation
Soundcheck (System of Record)
Event
Timeline
People
Payments
Assets
Venue
Loop/Observability